

Machine Learning Models: From Clustering to Reinforcement Learning

CS181 Study Guide

Contents

1	Clustering	3
1.1	K-Means Clustering	3
1.1.1	Purpose	3
1.1.2	Parameters	3
1.1.3	Training Method: Lloyd's Algorithm	3
1.1.4	Inference	3
1.1.5	Special Considerations	4
1.1.6	The Story of K-Means	4
1.2	Hierarchical Agglomerative Clustering (HAC)	4
1.2.1	Purpose	4
1.2.2	Parameters	4
1.2.3	Training Method	4
1.2.4	Linkage Criteria	4
1.2.5	Inference	5
1.2.6	Special Considerations	5
1.2.7	The Story of HAC	5
2	Dimensionality Reduction	5
2.1	Principal Component Analysis (PCA)	5
2.1.1	Purpose	5
2.1.2	Parameters	5
2.1.3	Training Method	5
2.1.4	Inference	6
2.1.5	Special Considerations	6
2.1.6	The Story of PCA	6
3	Graphical Models	7
3.1	Directed Graphical Models (Bayesian Networks)	7
3.1.1	Purpose	7
3.1.2	Parameters	7
3.1.3	Model Components	7
3.1.4	Model Creation	7
3.1.5	Inference	7
3.1.6	D-Separation and Independence	8
3.1.7	Special Considerations	8
3.1.8	The Story of Bayesian Networks	8

4	Mixture Models	8
4.1	Expectation-Maximization (EM) Algorithm	8
4.1.1	Purpose	8
4.1.2	Parameters	8
4.1.3	Training Method	9
4.1.4	Inference	9
4.1.5	Special Considerations	9
4.1.6	The Story of EM	9
4.2	Gaussian Mixture Models (GMMs)	9
4.2.1	Purpose	9
4.2.2	Parameters	9
4.2.3	Training Method	10
4.2.4	Inference	10
4.2.5	Connection to K-means	10
4.2.6	The Story of GMMs	10
4.3	Latent Dirichlet Allocation (LDA)	10
4.3.1	Purpose	10
4.3.2	Parameters	10
4.3.3	Generative Process	11
4.3.4	Training with EM	11
4.3.5	Inference	11
4.3.6	Special Considerations	11
4.3.7	The Story of LDA	11
5	Hidden Markov Models	12
5.1	Hidden Markov Models (HMMs)	12
5.1.1	Purpose	12
5.1.2	Parameters	12
5.1.3	HMM Assumptions	12
5.1.4	Forward-Backward Algorithm	12
5.1.5	Inference Tasks	13
5.1.6	Training with EM	13
5.1.7	Special Considerations	13
5.1.8	The Story of HMMs	14
6	Markov Decision Processes	14
6.1	Markov Decision Processes (MDPs)	14
6.1.1	Purpose	14
6.1.2	Components	14
6.1.3	Objective	14
6.1.4	Value Functions	14
6.1.5	Bellman Equations	15
6.1.6	Planning Algorithms	15
6.1.7	Special Considerations	15
6.1.8	The Story of MDPs	16
7	Reinforcement Learning	16
7.1	Reinforcement Learning (RL)	16
7.1.1	Purpose	16
7.1.2	Key Components	16
7.1.3	The Exploration-Exploitation Tradeoff	16
7.1.4	Model-Free vs. Model-Based RL	16
7.1.5	Value-Based Methods	17
7.1.6	Policy-Based Methods	17

7.1.7	Convergence Conditions	17
7.1.8	Special Considerations	17
7.1.9	The Story of Reinforcement Learning	17

1 Clustering

1.1 K-Means Clustering

1.1.1 Purpose

K-Means clustering is an unsupervised learning technique that identifies K groups (clusters) in data based on similarity. It minimizes the within-cluster variance, grouping data points that are close to each other.

1.1.2 Parameters

- K : The number of clusters
- $\{\mu_c\}_{c=1}^K$: The K cluster centers in D -dimensional space
- $\{r_n\}_{n=1}^N$: Responsibility vectors (one-hot encoded) assigning each data point to a cluster

1.1.3 Training Method: Lloyd's Algorithm

K-Means uses coordinate descent through Lloyd's algorithm:

1. **Initialize** K cluster centers randomly from data points

2. **Repeat until convergence:**

(a) **Assignment step:** Assign each data point to the closest cluster center

$$r_{nc} = \begin{cases} 1 & \text{if } c = \arg \min_{c'} \|x_n - \mu_{c'}\|^2 \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

(b) **Update step:** Recalculate cluster centers as the mean of assigned points

$$\mu_c = \frac{\sum_{n=1}^N r_{nc} x_n}{\sum_{n=1}^N r_{nc}} \quad (2)$$

The objective function being minimized is:

$$L(X, \{\mu_c\}_{c=1}^C, \{r_n\}_{n=1}^N) = \sum_{n=1}^N \sum_{c=1}^C r_{nc} \|x_n - \mu_c\|_2^2 \quad (3)$$

1.1.4 Inference

For a new data point x_* , we compute the closest cluster center:

$$c_* = \arg \min_c \|x_* - \mu_c\|^2 \quad (4)$$

Questions answerable through K-means:

- Which group does a new data point belong to?
- What is the structure/grouping of my data?
- What are the representative centers of each group?

1.1.5 Special Considerations

- Converges to a local optimum, so multiple random initializations are recommended
- Sensitive to initial cluster center placement (K-means++ can improve this)
- Assumes spherical clusters of similar size
- The number of clusters K must be specified in advance (the "elbow method" can help determine this)
- Data should be standardized to prevent features with larger scales from dominating

1.1.6 The Story of K-Means

Imagine you have a field full of colored balls scattered randomly. You want to organize them into K piles based on their location. You start by picking K random positions as initial pile locations. Then you follow two simple steps repeatedly: (1) Assign each ball to its closest pile, and (2) Move each pile to the average position of all balls assigned to it. You continue this process until the piles stop moving significantly. This is exactly how K-means works.

1.2 Hierarchical Agglomerative Clustering (HAC)

1.2.1 Purpose

HAC builds a hierarchy of clusters by progressively merging the closest clusters, allowing for a multi-level understanding of data structure without requiring a pre-specified number of clusters.

1.2.2 Parameters

- Linkage criterion: Function that determines the distance between clusters
- Dendrogram: Tree structure showing the hierarchical relationship between clusters

1.2.3 Training Method

1. Start with each data point as its own cluster
2. Repeat until only one cluster remains:
 - (a) Compute distances between all pairs of clusters
 - (b) Merge the two closest clusters
 - (c) Record the distance at which clusters were merged

1.2.4 Linkage Criteria

- **Min-linkage:**

$$d_{C,C'} = \min_{k,k'} \|x_k - x_{k'}\| \quad (5)$$

where x_k are data points in cluster C and $x_{k'}$ are data points in cluster C'

- **Max-linkage:**

$$d_{C,C'} = \max_{k,k'} \|x_k - x_{k'}\| \quad (6)$$

- **Average-linkage:**

$$d_{C,C'} = \frac{1}{KK'} \sum_{k=1}^K \sum_{k'=1}^{K'} \|x_k - x_{k'}\| \quad (7)$$

- **Centroid-linkage:**

$$d_{C,C'} = \left\| \frac{1}{K} \sum_{k=1}^K x_k - \frac{1}{K'} \sum_{k'=1}^{K'} x_{k'} \right\| \quad (8)$$

1.2.5 Inference

After building the dendrogram, we can:

- Cut the dendrogram at a specific height to get a certain number of clusters
- For a new data point, find the closest cluster based on the chosen linkage criterion

Questions answerable:

- What is the hierarchical structure of my data?
- How many natural clusters exist in my data?
- What is the proximity relationship between different clusters?

1.2.6 Special Considerations

- Different linkage criteria produce different clustering results
- Min-linkage tends to create "stringy" clusters
- Max-linkage tends to create more compact, balanced clusters
- HAC is deterministic (unlike K-means) but computationally intensive ($O(N^2)$ space and $O(N^3)$ time)

1.2.7 The Story of HAC

Imagine you're organizing a social network. Initially, everyone is a separate individual (their own cluster). You start connecting people who are closest to each other, gradually forming larger social groups. Eventually, everyone becomes connected in a single network. The sequence and distances at which these connections form tell a story about the community structure. By cutting the tree at different levels, you can see different grouping granularities - from tight-knit friend circles to broader communities.

2 Dimensionality Reduction

2.1 Principal Component Analysis (PCA)

2.1.1 Purpose

PCA reduces the dimensionality of data while preserving as much variance as possible, enabling visualization, noise reduction, and computational efficiency for high-dimensional data.

2.1.2 Parameters

- Principal components (eigenvectors of the covariance matrix)
- Explained variance per component
- Number of components to retain D' (where $D' < D$)

2.1.3 Training Method

1. Center the data by subtracting the mean of each feature
2. Compute the covariance matrix $S = \frac{X^T X}{N}$
3. Find the eigenvectors and eigenvalues of S
4. Sort eigenvectors by decreasing eigenvalues

5. Select top D' eigenvectors to form the projection matrix

The optimization objective for PCA can be expressed in two equivalent ways:

1. **Variance maximization:** Find directions of maximum variance

$$\max_{\mathbf{w}: \|\mathbf{w}\|=1} \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n \cdot \mathbf{w})^2 = \max_{\mathbf{w}: \|\mathbf{w}\|=1} \mathbf{w}^T S \mathbf{w} \quad (9)$$

2. **Reconstruction loss minimization:** Minimize the error from projecting onto a subspace

$$\min_{\mathbf{w}: \|\mathbf{w}\|=1} \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - (\mathbf{x}_n \cdot \mathbf{w})\mathbf{w}\|^2 \quad (10)$$

2.1.4 Inference

For a new data point $\mathbf{x}_*$:

- Project onto reduced space: $\mathbf{z}_* = W^T(\mathbf{x}_* - \boldsymbol{\mu})$ where W is the matrix of principal components
- Reconstruct from reduced space: $\hat{\mathbf{x}}_* = W\mathbf{z}_* + \boldsymbol{\mu}$

Questions answerable:

- What are the most important dimensions/directions in my data?
- How can I visualize high-dimensional data in 2D or 3D?
- How much information is retained when reducing to D' dimensions?
- What is a compact representation of my data?

2.1.5 Special Considerations

- PCA assumes linear relationships between variables
- It works best when the principal components are orthogonal to each other
- The choice of D' can be based on:
 - Cumulative explained variance (e.g., retain components that explain 95% of variance)
 - Scree plot ("elbow method")
 - Specific needs (e.g., visualization in 2D or 3D)
- Data should be standardized if features have different scales

2.1.6 The Story of PCA

Imagine you're studying a cloud of data points floating in a high-dimensional space. Looking carefully, you notice that although the cloud exists in many dimensions, it's actually quite flat - like a pancake. PCA helps you find the directions along which the pancake extends the most. By focusing on these main directions and ignoring the others where little variation occurs, you can describe the pancake with fewer numbers while preserving its essential shape.

3 Graphical Models

3.1 Directed Graphical Models (Bayesian Networks)

3.1.1 Purpose

Directed graphical models (DGMs) or Bayesian Networks represent the conditional independence structure of a joint probability distribution, enabling efficient inference, knowledge representation, and reasoning under uncertainty.

3.1.2 Parameters

- Graph structure (nodes and directed edges)
- Conditional probability tables (CPTs) or distributions for each node given its parents

3.1.3 Model Components

- **Nodes:** Random variables (drawn as circles; shaded if observed)
- **Edges:** Direct dependencies between variables
- **Parameters:** Deterministic values (drawn as small dots)
- **Plates:** Repeated sets of variables (rectangles with a number in the corner)

3.1.4 Model Creation

For a set of variables, a Bayesian network can be constructed by:

1. Choosing a variable ordering
2. For each variable, drawing edges from previously added variables that directly influence it
3. Defining the conditional probability distribution for each node given its parents

The joint distribution factorizes according to the graph structure:

$$p(X_1, X_2, \dots, X_n) = \prod_{i=1}^n p(X_i | \text{Parents}(X_i)) \quad (11)$$

3.1.5 Inference

Inference involves calculating:

- Marginal probabilities: $p(X_i)$
- Conditional probabilities: $p(X_i | X_j = x_j)$
- Most probable explanation (MPE): $\arg \max_{x_1, \dots, x_n} p(X_1 = x_1, \dots, X_n = x_n | E = e)$ where E is evidence

Inference methods include:

- Variable elimination
- Message passing algorithms
- Sampling methods

3.1.6 D-Separation and Independence

D-separation rules determine conditional independence in a graph:

1. **Chain:** $A \rightarrow B \rightarrow C$
 - A and C are dependent
 - A and C are conditionally independent given B
2. **Fork:** $A \leftarrow B \rightarrow C$
 - A and C are dependent
 - A and C are conditionally independent given B
3. **Collider:** $A \rightarrow B \leftarrow C$
 - A and C are independent
 - A and C are conditionally dependent given B (or any descendant of B)

3.1.7 Special Considerations

- Different variable orderings can lead to different network structures
- Choosing a good elimination order is crucial for efficient inference
- The complexity of representing a distribution grows with the number of parents per node
- Bayesian networks can represent both causal and non-causal relationships

3.1.8 The Story of Bayesian Networks

Imagine a detective trying to solve a crime. There are multiple clues, suspects, and evidence pieces, all connected in a complex web of relationships. Some evidence directly affects the likelihood of certain scenarios, while other connections are indirect. A Bayesian network is like the detective's case board with pins (variables) and strings (dependencies) showing how everything relates. By understanding these relationships, the detective can make deductions about unobserved events based on the evidence at hand.

4 Mixture Models

4.1 Expectation-Maximization (EM) Algorithm

4.1.1 Purpose

The EM algorithm finds maximum likelihood parameters in models with latent variables, where direct optimization would be intractable. It's the primary method for training mixture models.

4.1.2 Parameters

- Model parameters θ (specific to the mixture model being used)
- Latent variables Z (typically cluster assignments)
- Responsibilities q_n (soft assignments of data points to clusters)

4.1.3 Training Method

1. **Initialize** parameters θ randomly
2. **Repeat until convergence:**
 - (a) **E-step:** Compute the expected values of latent variables Z (responsibilities)

$$q_n = E[z_n|x_n] = p(z_n|x_n; \theta) \quad (12)$$

- (b) **M-step:** Update parameters θ to maximize the expected complete-data log likelihood

$$\theta \in \arg \max_{\theta} E_{Z|X}[\log p(X, Z; \theta)] \quad (13)$$

The EM algorithm effectively maximizes the Evidence Lower Bound (ELBO):

$$\text{ELBO}(\theta, q) = \sum_{n=1}^N E_{z_n \sim q(z_n)}[\log p(x_n, z_n|\theta)] - \sum_{n=1}^N E_{z_n \sim q(z_n)}[\log q(z_n)] \quad (14)$$

4.1.4 Inference

For a new data point x_* , we can compute:

- Probability of being in each cluster: $p(z_*|x_*)$
- Most likely cluster: $\arg \max_k p(z_* = k|x_*)$
- Predictive likelihood: $p(x_*)$

4.1.5 Special Considerations

- EM guarantees increasing the likelihood at each iteration but may converge to a local optimum
- Multiple random initializations are recommended
- The number of components/clusters must typically be specified in advance
- EM is a coordinate ascent algorithm: it alternates between optimizing q and θ

4.1.6 The Story of EM

Imagine trying to decipher a document written in an unknown language where each word could come from one of several possible languages. Initially, you make random guesses about which language each word comes from. Then you alternate between two steps: (1) for each word, compute the probability it comes from each language based on your current language models; (2) update your model of each language based on these probabilistic assignments. By repeating this process, you gradually improve both your language models and your word classifications.

4.2 Gaussian Mixture Models (GMMs)

4.2.1 Purpose

GMMs model data as being generated from a mixture of several Gaussian distributions, providing a flexible way to model complex, multimodal distributions.

4.2.2 Parameters

- K : Number of Gaussian components
- $\theta = \{\theta_k\}_{k=1}^K$: Mixture weights (prior probabilities of each component)
- $\{\mu_k, \Sigma_k\}_{k=1}^K$: Means and covariance matrices of each Gaussian component

4.2.3 Training Method

GMMs are trained using the EM algorithm:

E-step: Compute responsibilities

$$q_{nk} = p(z_n = k | x_n) = \frac{\theta_k \mathcal{N}(x_n; \mu_k, \Sigma_k)}{\sum_{j=1}^K \theta_j \mathcal{N}(x_n; \mu_j, \Sigma_j)} \quad (15)$$

M-step: Update parameters

$$\theta_k = \frac{\sum_{n=1}^N q_{nk}}{N} \quad (16)$$

$$\mu_k = \frac{\sum_{n=1}^N q_{nk} x_n}{\sum_{n=1}^N q_{nk}} \quad (17)$$

$$\Sigma_k = \frac{\sum_{n=1}^N q_{nk} (x_n - \mu_k)(x_n - \mu_k)^T}{\sum_{n=1}^N q_{nk}} \quad (18)$$

4.2.4 Inference

For a new data point x_* :

- Cluster membership probabilities: $p(z_* = k | x_*) \propto \theta_k \mathcal{N}(x_*; \mu_k, \Sigma_k)$
- Density estimation: $p(x_*) = \sum_{k=1}^K \theta_k \mathcal{N}(x_*; \mu_k, \Sigma_k)$
- Anomaly detection: Points with low $p(x_*)$ can be considered anomalies

4.2.5 Connection to K-means

GMMs generalize K-means clustering:

- K-means assumes spherical clusters of equal size
- GMMs allow elliptical clusters of varying sizes and orientations
- K-means makes hard assignments; GMMs make soft probabilistic assignments
- As covariances approach zero, GMM approaches K-means

4.2.6 The Story of GMMs

Imagine studying the heights of a mixed population of adults and children. The distribution might have two "bumps" - one centered around child heights and another around adult heights. A GMM would model this as a mixture of two Gaussian distributions, each with its own mean and variance. For a new person, the model could tell you the probability they belong to either the child or adult group, based on their height.

4.3 Latent Dirichlet Allocation (LDA)

4.3.1 Purpose

LDA is a generative probabilistic model for collections of discrete data such as text corpora. It represents documents as mixtures of topics and each topic as a distribution over words.

4.3.2 Parameters

- α : Dirichlet prior on document-topic distributions
- β : Dirichlet prior on topic-word distributions
- θ_n : Topic mixture for document n
- ϕ_k : Word distribution for topic k

4.3.3 Generative Process

1. For each document n :
 - (a) Draw topic proportions $\theta_n \sim \text{Dir}(\alpha)$
 - (b) For each word position j in document n :
 - i. Draw topic assignment $z_{n,j} \sim \text{Cat}(\theta_n)$
 - ii. Draw word $w_{n,j} \sim \text{Cat}(\phi_{z_{n,j}})$
2. For each topic k :
 - (a) Draw word distribution $\phi_k \sim \text{Dir}(\beta)$

4.3.4 Training with EM

E-step: Compute the posterior distribution over topic assignments

$$q_{n,j} = p(z_{n,j}|w_{n,j}) \propto \phi_{z_{n,j},w_{n,j}} \cdot \theta_{n,z_{n,j}} \quad (19)$$

M-step: Update parameters

$$\theta_{n,k} \propto \alpha_k + \sum_{j=1}^J q_{n,j,k} \quad (20)$$

$$\phi_{k,w} \propto \beta_w + \sum_{n=1}^N \sum_{j=1}^{J_n} q_{n,j,k} \cdot \mathbf{1}[w_{n,j} = w] \quad (21)$$

4.3.5 Inference

LDA can answer questions like:

- What are the main topics in a document collection?
- What is the topic distribution of a specific document?
- Which documents are most similar based on topic distribution?
- What words are most characteristic of each topic?

4.3.6 Special Considerations

- The number of topics must be specified in advance
- The choice of hyperparameters α and β affects the sparsity of topic and word distributions
- LDA assumes bag-of-words representation (word order doesn't matter)
- LDA is a true "admixture" model, meaning each document can belong to multiple topics

4.3.7 The Story of LDA

Imagine a library with books from different subjects. Each book contains a mixture of topics - a cookbook might be 70

5 Hidden Markov Models

5.1 Hidden Markov Models (HMMs)

5.1.1 Purpose

HMMs model sequential data with hidden states, where observations depend on the current hidden state, and state transitions follow the Markov property. They're useful for speech recognition, natural language processing, and bioinformatics.

5.1.2 Parameters

- $\theta \in [0, 1]^K$: Initial state distribution, where $\sum_k \theta_k = 1$
- $T \in [0, 1]^{K \times K}$: Transition matrix, where $T_{i,j} = p(s_{t+1} = j | s_t = i)$
- $\pi \in [0, 1]^{K \times M}$: Emission matrix, where $\pi_{k,m} = p(x_t = m | s_t = k)$

5.1.3 HMM Assumptions

1. **Markov property:** The current state depends only on the previous state

$$p(s_{t+1} | s_1, \dots, s_t, x_1, \dots, x_t) = p(s_{t+1} | s_t) \quad (22)$$

2. **Output independence:** The current observation depends only on the current state

$$p(x_t | s_1, \dots, s_t, x_1, \dots, x_{t-1}) = p(x_t | s_t) \quad (23)$$

These assumptions allow us to factorize the joint distribution:

$$p(s_1, \dots, s_n, x_1, \dots, x_n) = p(s_1) \prod_{t=1}^{n-1} p(s_{t+1} | s_t) \prod_{t=1}^n p(x_t | s_t) \quad (24)$$

5.1.4 Forward-Backward Algorithm

The forward-backward algorithm efficiently computes probabilities for HMM inference. It defines:

Forward messages (α):

$$\alpha_t(s_t) = p(x_1, \dots, x_t, s_t) \quad (25)$$

Recursive computation:

$$\alpha_t(s_t) = \begin{cases} p(x_t | s_t) \sum_{s_{t-1}} p(s_t | s_{t-1}) \alpha_{t-1}(s_{t-1}) & \text{if } 1 < t \leq n \\ p(x_1 | s_1) p(s_1) & \text{otherwise} \end{cases} \quad (26)$$

Backward messages (β):

$$\beta_t(s_t) = p(x_{t+1}, \dots, x_n | s_t) \quad (27)$$

Recursive computation:

$$\beta_t(s_t) = \begin{cases} \sum_{s_{t+1}} p(s_{t+1} | s_t) p(x_{t+1} | s_{t+1}) \beta_{t+1}(s_{t+1}) & \text{if } 1 \leq t < n \\ 1 & \text{otherwise} \end{cases} \quad (28)$$

5.1.5 Inference Tasks

Using α and β values, we can perform various inference tasks:

Sequence probability:

$$p(x_1, \dots, x_n) = \sum_{s_t} \alpha_t(s_t) \beta_t(s_t) \quad (29)$$

Filtering (current state given observations up to now):

$$p(s_t | x_1, \dots, x_t) \propto \alpha_t(s_t) \quad (30)$$

Smoothing (state at time t given all observations):

$$p(s_t | x_1, \dots, x_n) \propto \alpha_t(s_t) \beta_t(s_t) \quad (31)$$

Prediction (future observation given past observations):

$$p(x_{t+1} | x_1, \dots, x_t) \propto \sum_{s_t} \sum_{s_{t+1}} \alpha_t(s_t) p(s_{t+1} | s_t) p(x_{t+1} | s_{t+1}) \quad (32)$$

Viterbi algorithm (most likely state sequence): Define $\gamma_t(s_t) = \max_{s_1, \dots, s_{t-1}} p(s_1, \dots, s_t, x_1, \dots, x_t)$

$$\gamma_t(s_t) = \begin{cases} p(x_t | s_t) \max_{s_{t-1}} p(s_t | s_{t-1}) \gamma_{t-1}(s_{t-1}) & \text{if } 1 < t \leq n \\ p(x_1 | s_1) p(s_1) & \text{otherwise} \end{cases} \quad (33)$$

Also maintain backpointers:

$$z_t^*(s_t) = \arg \max_{s_{t-1}} [p(s_t | s_{t-1}) \gamma_{t-1}(s_{t-1})] \quad (34)$$

5.1.6 Training with EM

HMMs are trained using EM (also called the Baum-Welch algorithm in this context):

E-step: Compute posterior probabilities of states and transitions using forward-backward

$$q_{t,k} = p(s_t = k | x_1, \dots, x_n) = \frac{\alpha_t(s_t = k) \beta_t(s_t = k)}{\sum_j \alpha_t(s_t = j) \beta_t(s_t = j)} \quad (35)$$

$$Q_{t,t+1,k,l} = p(s_t = k, s_{t+1} = l | x_1, \dots, x_n) \quad (36)$$

M-step: Update parameters

$$\theta_k = \frac{\sum_{i=1}^N q_{1,k}^i}{N} \quad (37)$$

$$T_{k,l} = \frac{\sum_{i=1}^N \sum_{t=1}^{n-1} Q_{t,t+1,k,l}^i}{\sum_{i=1}^N \sum_{t=1}^{n-1} q_{t,k}^i} \quad (38)$$

$$\pi_{k,m} = \frac{\sum_{i=1}^N \sum_{t=1}^n q_{t,k}^i x_{t,m}^i}{\sum_{i=1}^N \sum_{t=1}^n q_{t,k}^i} \quad (39)$$

5.1.7 Special Considerations

- HMMs are generative models for sequential data
- They assume the Markov property (limited memory)
- Forward-backward algorithm complexity is $O(K^2 n)$ where K is the number of states and n is sequence length
- HMMs can only model discrete state spaces effectively (for continuous states, see Kalman filters)

5.1.8 The Story of HMMs

Imagine a weather forecaster who can't see outside but receives daily reports about whether people carried umbrellas. From these observations, the forecaster tries to deduce whether it was sunny, cloudy, or rainy each day. The actual weather states are hidden, while umbrella usage is observed. The forecaster knows that weather tends to be similar on consecutive days (the Markov property) and that people are more likely to carry umbrellas when it rains. Using these patterns, the forecaster can reconstruct the most likely weather sequence and predict future weather - exactly what an HMM does.

6 Markov Decision Processes

6.1 Markov Decision Processes (MDPs)

6.1.1 Purpose

MDPs provide a mathematical framework for modeling decision-making in situations where outcomes are partly random and partly under the control of a decision-maker (agent). They're fundamental to reinforcement learning and planning under uncertainty.

6.1.2 Components

- S : Set of states
- A : Set of actions
- $p(s'|s, a)$: Transition function defining the probability of reaching state s' from state s by taking action a
- $r(s, a)$: Reward function defining the immediate reward for taking action a in state s
- $\gamma \in [0, 1]$: Discount factor for future rewards
- $\pi(a|s)$: Policy mapping states to actions

6.1.3 Objective

The goal is to find an optimal policy π^* that maximizes the expected discounted total reward:

$$\max_{\pi} E \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] \quad (40)$$

6.1.4 Value Functions

State-value function (value of being in state s following policy π):

$$V^{\pi}(s) = E_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, \pi(s_t)) | s_0 = s \right] \quad (41)$$

Action-value function (value of taking action a in state s following policy π):

$$Q^{\pi}(s, a) = E_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) | s_0 = s, a_0 = a \right] \quad (42)$$

Optimal value functions:

$$V^*(s) = \max_{\pi} V^{\pi}(s) \quad (43)$$

$$Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a) \quad (44)$$

6.1.5 Bellman Equations

Bellman Consistency Equation (for a policy π):

$$V^\pi(s) = r(s, \pi(s)) + \gamma \sum_{s'} p(s'|s, \pi(s)) V^\pi(s') \quad (45)$$

Bellman Optimality Equation:

$$V^*(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V^*(s') \right] \quad (46)$$

Bellman Operator $B : \mathbb{R}^{|S|} \rightarrow \mathbb{R}^{|S|}$:

$$B(V)(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V(s') \right] \quad (47)$$

6.1.6 Planning Algorithms

Value Iteration:

1. Initialize $V(s) = 0$ for all $s \in S$
2. Repeat until convergence:

$$V'(s) \leftarrow \max_a \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V(s') \right] \quad \forall s \quad (48)$$

$$V(s) \leftarrow V'(s) \quad \forall s \quad (49)$$

Policy Iteration:

1. Initialize policy π arbitrarily
2. Repeat until convergence:
 - (a) **Policy evaluation:** Calculate $V^\pi(s)$ for all s
 - (b) **Policy improvement:**

$$\pi'(s) \leftarrow \arg \max_a \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V^\pi(s') \right] \quad \forall s \quad (50)$$

$$\pi(s) \leftarrow \pi'(s) \quad \forall s \quad (51)$$

Policy Evaluation (exact):

$$V^\pi = (I - \gamma P^\pi)^{-1} R^\pi \quad (52)$$

where P^π is the transition matrix induced by policy π and R^π is the reward vector.

6.1.7 Special Considerations

- Value iteration has time complexity $O(|S|^2|A|)$ per iteration
- Policy iteration often converges in fewer iterations but each iteration costs $O(|S|^2|A| + |S|^3)$
- The discount factor γ balances immediate vs. future rewards:
 - $\gamma \rightarrow 0$: Myopic, only immediate rewards matter
 - $\gamma \rightarrow 1$: Far-sighted, future rewards valued nearly as much as immediate ones
- The effective time horizon is approximately $\frac{1}{1-\gamma}$

6.1.8 The Story of MDPs

Imagine a treasure hunter in a maze. At each intersection (state), they can choose which path to take (action). Some paths lead to small treasures, others to traps, and eventually, they hope to find the big treasure. The maze is foggy, so when they choose a path, they don't always end up where they intended (stochastic transitions). The hunter wants to find a strategy (policy) for navigating the maze that will maximize their total treasure. MDPs provide the mathematical framework to determine this optimal strategy, balancing immediate rewards against long-term goals.

7 Reinforcement Learning

7.1 Reinforcement Learning (RL)

7.1.1 Purpose

Reinforcement learning is about learning how an agent should take actions in an environment to maximize cumulative reward when the environment's dynamics (transition probabilities and rewards) are unknown.

7.1.2 Key Components

- **Agent:** The learner making decisions
- **Environment:** The world the agent interacts with
- **State s_t :** The current situation the agent is in
- **Action a_t :** The decision the agent makes
- **Reward r_t :** Feedback from the environment after taking an action
- **Policy $\pi(a|s)$:** Strategy for selecting actions

7.1.3 The Exploration-Exploitation Tradeoff

A central challenge in RL is balancing:

- **Exploration:** Trying new actions to discover their outcomes
- **Exploitation:** Using known good actions to maximize reward

One common approach is the ϵ -greedy strategy:

$$\pi(s) = \begin{cases} \arg \max_a Q(s, a) & \text{with probability } 1 - \epsilon \\ \text{random action} & \text{with probability } \epsilon \end{cases} \quad (53)$$

7.1.4 Model-Free vs. Model-Based RL

Model-Free RL:

- Learns value functions or policies directly from experience
- Doesn't explicitly model the environment's dynamics
- Examples: Q-learning, SARSA, Policy Gradient

Model-Based RL:

- Learns a model of the environment (transition and reward functions)
- Uses this model for planning (e.g., using value iteration)
- Can be more sample-efficient but potentially less accurate if the model is wrong

7.1.5 Value-Based Methods

Temporal Difference (TD) Learning: Updates value estimates based on the difference between successive predictions:

$$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)] \quad (54)$$

where α is the learning rate and the term in brackets is the TD error.

SARSA (on-policy):

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)] \quad (55)$$

where a' is the action taken in state s' according to policy π .

Q-learning (off-policy):

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (56)$$

Deep Q-Networks (DQN): Uses neural networks to approximate Q-values:

$$L(w_i) = E[(r + \gamma \max_{a'} Q(s', a'; w_{i-1}) - Q(s, a; w_i))^2] \quad (57)$$

where w_i are the neural network parameters at iteration i .

7.1.6 Policy-Based Methods

Directly optimize the policy $\pi_\theta(a|s)$ parameterized by θ :

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta) \quad (58)$$

The gradient of the performance measure can be approximated as:

$$\nabla_\theta J(\theta) \approx E \left[\sum_t \nabla_\theta \ln \pi_\theta(a_t|s_t) r_t \right] \quad (59)$$

7.1.7 Convergence Conditions

For Q-learning to converge to Q^* :

- $\sum_t \alpha_t(s, a) = \infty$ for all (s, a) pairs
- $\sum_t \alpha_t(s, a)^2 < \infty$ for all (s, a) pairs
- All state-action pairs must be visited infinitely often

For SARSA, the policy must also become greedy in the limit ($\epsilon \rightarrow 0$ as $t \rightarrow \infty$).

7.1.8 Special Considerations

- RL algorithms can be unstable, especially when using function approximation
- Experience replay can make learning more stable by breaking correlations in sequential data
- Bootstrapping (using estimates to update estimates) can lead to instability
- The choice between on-policy and off-policy methods depends on the specific problem

7.1.9 The Story of Reinforcement Learning

Imagine a child learning to ride a bicycle. Initially, they have no idea how to balance or pedal effectively. Through trial and error, they learn which actions lead to successful riding and which result in falls. They first explore different techniques, gradually exploiting the ones that work best. The reward is staying upright, and the punishment is falling. Over time, they develop an intuitive policy for riding that maximizes their success. Reinforcement learning formalizes this natural learning process, enabling machines to learn optimal behaviors through their own experiences.